Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 6
по дисциплине «Объектно-ориентированное программирование»

на тему «Делегаты и события в C#»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Приобрести навыки работы с делегатами, событиями, анонимными методами, лямбда-выражениями и стандартными обобщёнными делегатами (Action, Func, Predicate) в языке программирования C#.

2. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

В рамках лабораторной работы были решены следующие задачи:

1. Реализовать класс `TaskItem` (модель задачи)
2. Реализовать класс `TaskEventArgs` (аргументы события)
3. Реализовать класс `TaskManager` (управление задачами с событиями)
4. Реализовать собственные делегаты (`TaskFilter`, `NotifyHandler`)
5. Реализовать multicast-делегат для системы уведомлений
6. Применить стандартные делегаты (`Action<T>`, `Func<T>`)
7. Применить лямбда-выражения
8. Применить LINQ для фильтрации и статистики
9. Разработать графический интерфейс

5. ЛИСТИНГ ПРОГРАММЫ

Класс TaskItem (модель задачи):

```
using System;

namespace TaskManagerGUI
{
    public class TaskItem
    {
        public string Name { get; set; }
        public int Priority { get; set; }
        public bool IsDone { get; set; }
        public DateTime CreatedAt { get; set; }

        public TaskItem(string name, int priority)
        {
            Name = name;
            Priority = priority;
            IsDone = false;
            CreatedAt = DateTime.Now;
        }

        public override string ToString()
        {
            string status = IsDone ? "[✓]" : "[ ]";
            return $"{status} [{Priority}] {Name}";
        }
    }
}
```

Класс TaskEventArgs (аргументы события):

```
using System;

namespace TaskManagerGUI
{
    public class TaskEventArgs : EventArgs
    {
        public TaskItem Task { get; }
        public string Action { get; }
        public DateTime Time { get; }

        public TaskEventArgs(TaskItem task, string
action)
        {
            Task = task;
            Action = action;
            Time = DateTime.Now;
        }
    }
}
```

Класс TaskManager (управление задачами с событиями):

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace TaskManagerGUI
{
    public class TaskManager
    {
        private List<TaskItem> allTasks = new
List<TaskItem>();

        public event EventHandler<TaskEventArgs>?
TaskChanged;

        public void AddTask(TaskItem task)
        {
            allTasks.Add(task);
            TaskChanged?.Invoke(this, new
TaskEventArgs(task, "Добавлена"));
        }

        public void RemoveTask(TaskItem task)
        {
            allTasks.Remove(task);
            TaskChanged?.Invoke(this, new
TaskEventArgs(task, "Удалена"));
        }

        public void ToggleTaskStatus(TaskItem task)
        {
            task.IsDone = !task.IsDone;
            string action = task.IsDone ? "Выполнена" :
"Возвращена";
            TaskChanged?.Invoke(this, new
TaskEventArgs(task, action));
        }

        public List<TaskItem>
GetFilteredTasks(Func<TaskItem, bool> filter)
        {
            return allTasks
```

```
                .Where(filter)
                .OrderBy(t => t.IsDone)
                .ThenByDescending(t => t.Priority)
                .ToList();
        }

        public List<TaskItem> GetAllTasks() =>
allTasks;
    }
}
```

Графический интерфейс (MainWindow.axaml):

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/
2006/xaml"
        x:Class="TaskManagerGUI.Views.MainWindow"
        Title="Менеджер задач - ЛР6"
        Width="900" Height="600">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!-- Панель добавления -->
        <Border Grid.Row="0" BorderBrush="Gray"
BorderThickness="1" Padding="10" Margin="0,0,0,10">
            <StackPanel Orientation="Horizontal"
Spacing="10">
                <TextBlock Text="Название:"
VerticalAlignment="Center"/>
                <TextBox Name="textBoxName"
Width="250"/>
                <TextBlock Text="Приоритет:"
VerticalAlignment="Center"/>
                <NumericUpDown Name="numericPriority"
                               Minimum="1" Maximum="5"
Value="3" Width="100"
                               FormatString="0"
                               ShowButtonSpinner="True"
                               AllowSpin="True"

ButtonSpinnerLocation="Right"/>
                <Button Name="buttonAdd"
Content="Добавить" Click="OnAddClick" Width="100"/>
            </StackPanel>
        </Border>
```

```
        <!-- Основная область -->
        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="2*"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>

            <!-- Левая панель: задачи -->
            <Border Grid.Column="0" BorderBrush="Gray"
BorderThickness="1" Padding="10" Margin="0,0,5,0">
                <StackPanel>
                    <TextBlock Text="Список задач"
FontSize="16" FontWeight="Bold" Margin="0,0,0,10"/>
                    <ComboBox Name="comboBoxFilter"
                             SelectedIndex="0"

SelectionChanged="OnFilterChanged"
                             Margin="0,0,0,10">
                        <ComboBoxItem Content="Все
задачи"/>
                        <ComboBoxItem
Content="Выполненные"/>
                        <ComboBoxItem
Content="Невыполненные"/>
                        <ComboBoxItem Content="Высокий
приоритет (4-5)"/>
                    </ComboBox>
                    <ListBox Name="listBoxTasks"
Height="300" Margin="0,0,0,10"/>
                    <StackPanel
Orientation="Horizontal" Spacing="10">
                        <Button Name="buttonDone"
                               Content="Выполнено/
Вернуть"
                               Click="OnDoneClick"
                               Width="150"/>
                        <Button Name="buttonDelete"
                               Content="Удалить"
                               Click="OnDeleteClick"
                               Width="100"/>
```

```
                    </StackPanel>
                </StackPanel>
            </Border>

            <!-- Правая панель: журнал -->
            <Border Grid.Column="1" BorderBrush="Gray"
BorderThickness="1" Padding="10" Margin="5,0,0,0">
                <StackPanel>
                    <TextBlock Text="Журнал событий"
FontSize="16" FontWeight="Bold" Margin="0,0,0,10"/>
                    <ListBox Name="listBoxLog"
Height="300" Margin="0,0,0,10"/>
                    <CheckBox Name="checkBoxNotify"
                              Content="Включить
уведомления"
                              IsChecked="True"

IsCheckedChanged="OnNotifyCheckedChanged"/>
                </StackPanel>
            </Border>
        </Grid>

        <!-- Нижняя панель: статистика -->
        <Border Grid.Row="2" BorderBrush="Gray"
BorderThickness="1" Padding="10" Margin="0,10,0,0">
            <StackPanel>
                <TextBlock Name="labelStats"
                           FontSize="12"
                           FontWeight="Bold"
                           Margin="0,0,0,5"/>
                <TextBlock Name="labelStatus"
                           FontSize="11"
                           Foreground="DarkBlue"/>
            </StackPanel>
        </Border>
    </Grid>

</Window>
```

Логика интерфейса с делегатами (MainWindow.axaml.cs):

```
using Avalonia.Controls;
using Avalonia.Interactivity;
using System;
using System.Linq;

namespace TaskManagerGUI.Views
{
    public partial class MainWindow : Window
    {
        // Собственные делегаты (базовое задание)
        delegate bool TaskFilter(TaskItem task);
        delegate void NotifyHandler(string message);

        // Multicast-делегат для уведомлений
        private NotifyHandler? notifyHandlers;

        // Менеджер задач с событиями
        private TaskManager taskManager;

        // Обработчики событий для возможности отписки
        private EventHandler<TaskEventArgs>?
taskChangedHandler;
        private EventHandler<TaskEventArgs>?
taskChangedStatsHandler;

        public MainWindow()
        {
            InitializeComponent();

            // Инициализация менеджера
            taskManager = new TaskManager();

            // Создание multicast-делегата
            notifyHandlers = LogToListBox;
            notifyHandlers += ShowInStatusBar;

            // Подписка на события
            taskChangedHandler = OnTaskChanged;
```

```
            taskChangedStatsHandler =
OnTaskChangedStats;

            taskManager.TaskChanged +=
taskChangedHandler;
            taskManager.TaskChanged +=
taskChangedStatsHandler;

            // Инициализация
            UpdateStats();
        }

        // ==================== MULTICAST-ДЕЛЕГАТ
====================

        private void LogToListBox(string msg)
        {
            string logMsg = $"{DateTime.Now:HH:mm:ss} -
{msg}";
            listBoxLog.Items.Add(logMsg);

            // Автопрокрутка вниз
            if (listBoxLog.ItemCount > 0)
            {

listBoxLog.ScrollIntoView(listBoxLog.Items[listBoxLog.I
temCount - 1]!);
            }
        }

        private void ShowInStatusBar(string msg)
        {
            labelStatus.Text = $"Последнее действие:
{msg}";
        }

        // ==================== ФИЛЬТРЫ
====================
```

```
        // Методы фильтрации совместимые с делегатом
TaskFilter
        private bool FilterAll(TaskItem t) => true;
        private bool FilterDone(TaskItem t) =>
t.IsDone;
        private bool FilterNotDone(TaskItem t) => !
t.IsDone;
        private bool FilterHighPriority(TaskItem t) =>
t.Priority >= 4;

        private TaskFilter GetCurrentFilter()
        {
            // Защита от вызова до инициализации
            if (comboBoxFilter == null) return
FilterAll;

            return comboBoxFilter.SelectedIndex switch
            {
                1 => FilterDone,
                2 => FilterNotDone,
                3 => FilterHighPriority,
                _ => FilterAll
            };
        }

        // Применение фильтра с LINQ и лямбда-
выражениями
        private void ApplyFilter()
        {
            // Защита от вызова до инициализации
            if (listBoxTasks == null || taskManager ==
null) return;

            TaskFilter filter = GetCurrentFilter();

            // Получаем отфильтрованные и
отсортированные задачи через LINQ
            var filtered = taskManager.GetAllTasks()
                .Where(t => filter(t))
```

```
                .OrderBy(t => t.IsDone)           //
Сначала невыполненные
                .ThenByDescending(t => t.Priority) //
Затем по приоритету
                .ToList();

            listBoxTasks.Items.Clear();
            foreach (var task in filtered)
            {
                listBoxTasks.Items.Add(task);
            }
        }

        // ==================== СТАТИСТИКА С FUNC/
ACTION ====================

        private void UpdateStats()
        {
            // Защита от вызова до инициализации
            if (labelStats == null || taskManager ==
null) return;

            var tasks = taskManager.GetAllTasks();

            // Использование Func<T> для вычислений
            Func<int> getTotal = () => tasks.Count;
            Func<int> getDone = () => tasks.Count(t =>
t.IsDone);
            Func<int> getRemaining = () =>
tasks.Count(t => !t.IsDone);

            // Использование Action<string> для
обновления UI
            Action<string> updateLabel = text =>
labelStats.Text = text;

            int total = getTotal();
            int done = getDone();
            int remaining = getRemaining();
```

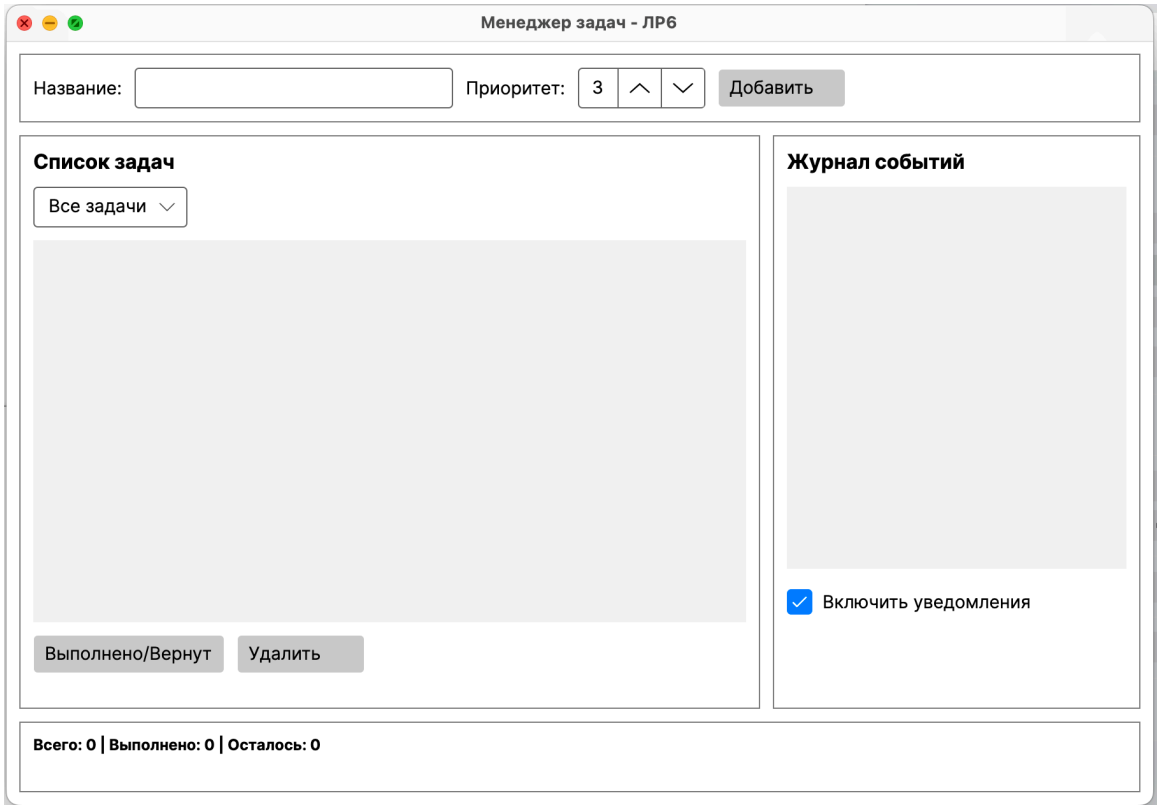
```
            // Расширенная статистика с LINQ
(усложненное задание)
            string statsText = $"Всего: {total} |
Выполнено: {done} | Осталось: {remaining}";

            if (total > 0)
            {
                double avgPriority = tasks.Average(t =>
(double)t.Priority);
                double donePercent = done * 100.0 /
total;

                var byPriority = tasks
                    .GroupBy(t => t.Priority)
                    .OrderBy(g => g.Key)
                    .Select(g => $"[{g.Key}]:
{g.Count()}")
                    .ToList();

                statsText += $" | Ср.приоритет:
{avgPriority:F1} | " +
                             $"Выполнено:
{donePercent:F0}% | " +
                             $"По приоритетам:
{string.Join(" ", byPriority)}";
            }

            updateLabel(statsText);
        }

        // ==================== ОБРАБОТЧИКИ СОБЫТИЙ
====================

        private void OnTaskChanged(object? sender,
TaskEventArgs e)
        {
            // Уведомления через multicast-делегат
            if (checkBoxNotify.IsChecked == true &&
notifyHandlers != null)
            {
```

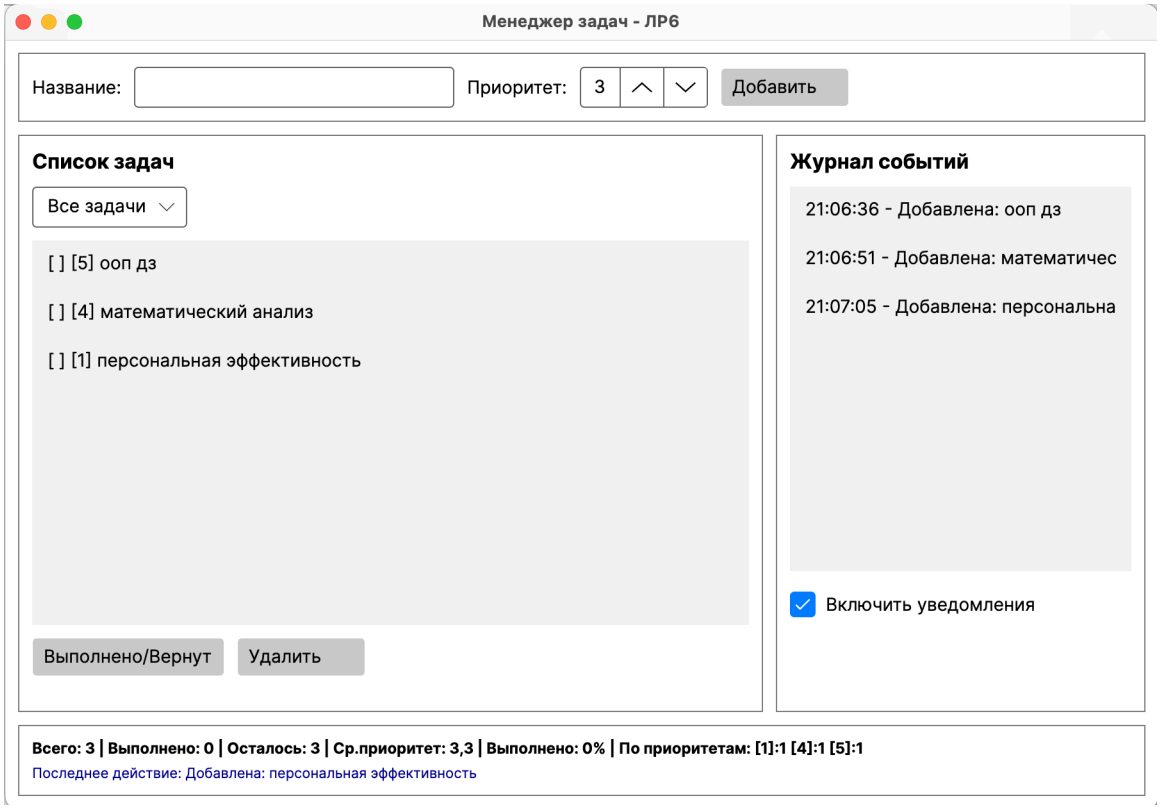
```
                notifyHandlers($"{e.Action}:
{e.Task.Name}");
            }
        }

        private void OnTaskChangedStats(object? sender,
TaskEventArgs e)
        {
            // Обновление статистики и списка
            UpdateStats();
            ApplyFilter();
        }

        // ==================== ОБРАБОТЧИКИ КНОПОК
====================

        private void OnAddClick(object? sender,
RoutedEventArgs e)
        {
            if
(string.IsNullOrWhiteSpace(textBoxName.Text))
            {
                labelStatus.Text = "Ошибка: Введите
название задачи!";
                return;
            }

            TaskItem task = new TaskItem(
                textBoxName.Text,
                (int)(numericPriority.Value ?? 3));

            taskManager.AddTask(task);
            textBoxName.Clear();
            numericPriority.Value = 3;
        }

        private void OnDeleteClick(object? sender,
RoutedEventArgs e)
        {
```

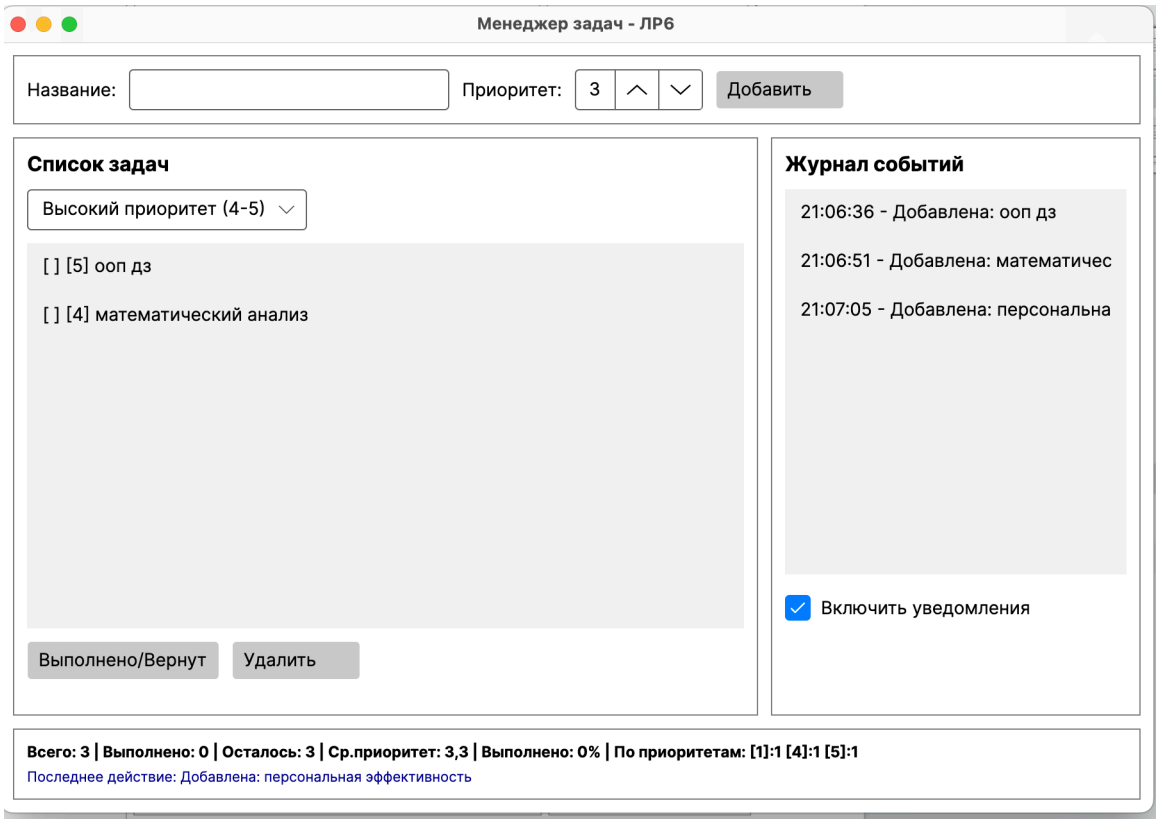
```
            if (listBoxTasks.SelectedItem is TaskItem
task)
            {
                taskManager.RemoveTask(task);
            }
            else
            {
                labelStatus.Text = "Ошибка: Выберите
задачу для удаления!";
            }
        }

        private void OnDoneClick(object? sender,
RoutedEventArgs e)
        {
            if (listBoxTasks.SelectedItem is TaskItem
task)
            {
                taskManager.ToggleTaskStatus(task);
            }
            else
            {
                labelStatus.Text = "Ошибка: Выберите
задачу!";
            }
        }

        private void OnFilterChanged(object? sender,
SelectionChangedEventArgs e)
        {
            // Защита от вызова до инициализации
            if (listBoxTasks == null || comboBoxFilter
== null) return;

            ApplyFilter();
        }

        // Обработчик CheckBox для включения/выключения
уведомлений (усложненное)
```

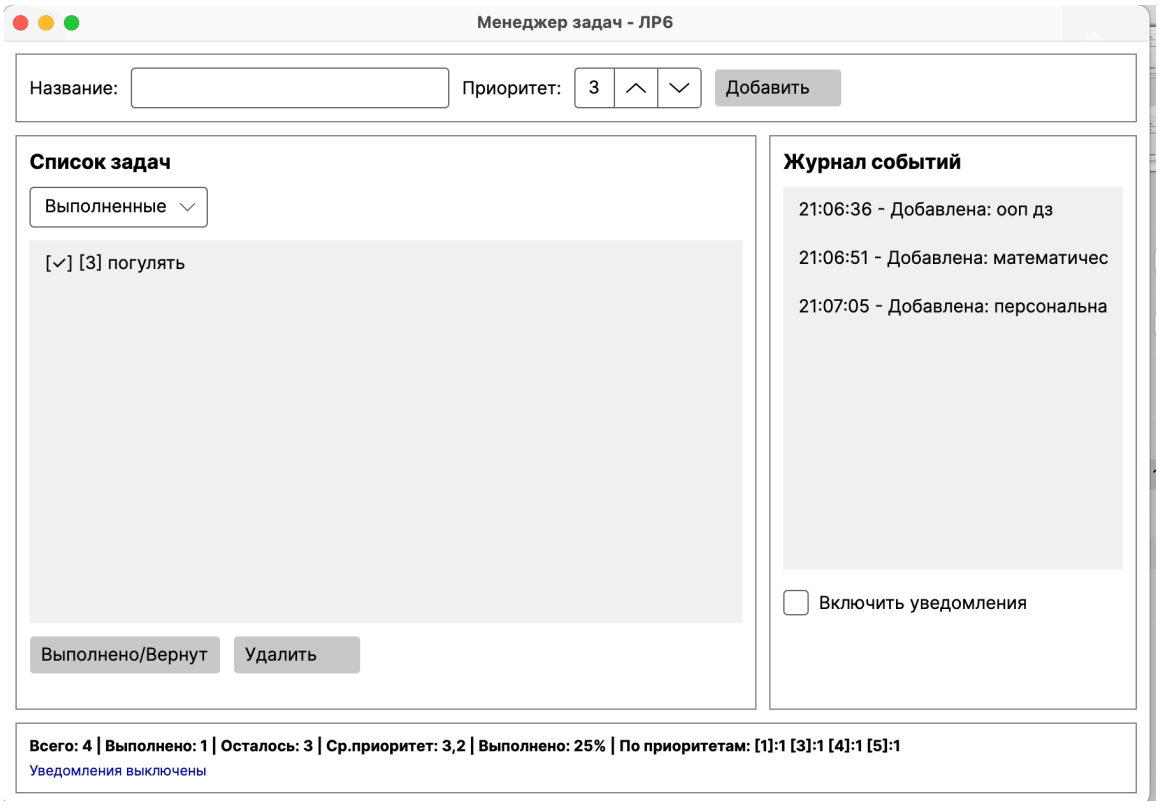
```
        private void OnNotifyCheckedChanged(object?
sender, RoutedEventArgs e)
        {
            if (labelStatus == null) return; // Защита
от вызова до инициализации

            if (checkBoxNotify.IsChecked == true)
            {
                // Подписываемся на события
                if (taskChangedHandler != null)
                {
                    taskManager.TaskChanged -=
taskChangedHandler;
                    taskManager.TaskChanged +=
taskChangedHandler;
                }
                labelStatus.Text = "Уведомления
включены";
            }
            else
            {
                labelStatus.Text = "Уведомления
выключены";
            }
        }
    }
}
```

Выполнение:


Менеджер задач - ЛР6
Название:
Приоритет:
3
Добавить
Список задач
Все задачи
Журнал событий
Включить уведомления
Выполнено/Вернут
Удалить
Всего: 0 | Выполнено: 0 | Осталось: 0


Менеджер задач - ЛР6
Название:
Приоритет:
3
Добавить
Список задач
Все задачи
[ ] [5] ооп дз
[ ] [4] математический анализ
[ ] [1] персональная эффективность
Журнал событий
21:06:36 - Добавлена: ооп дз
21:06:51 - Добавлена: математичес
21:07:05 - Добавлена: персональна
Включить уведомления
Выполнено/Вернут
Удалить
Всего: 3 | Выполнено: 0 | Осталось: 3 | Ср.приоритет: 3,3 | Выполнено: 0% | По приоритетам: [1]:1 [4]:1 [5]:1
Последнее действие: Добавлена: персональная эффективность


Менеджер задач - ЛР6
Название:
Приоритет:
3
Добавить
Список задач
Высокий приоритет (4-5)
[ ] [5] ооп дз
[ ] [4] математический анализ
Выполнено/Вернут
Удалить
Журнал событий
21:06:36 - Добавлена: ооп дз
21:06:51 - Добавлена: математичес
21:07:05 - Добавлена: персональна
Включить уведомления
Всего: 3 | Выполнено: 0 | Осталось: 3 | Ср.приоритет: 3,3 | Выполнено: 0% | По приоритетам: [1]:1 [4]:1 [5]:1
Последнее действие: Добавлена: персональная эффективность


Менеджер задач - ЛР6
Название:
Приоритет:
3
Добавить
Список задач
Выполненные
[✓] [3] погулять
Выполнено/Вернут
Удалить
Журнал событий
21:06:36 - Добавлена: ооп дз
21:06:51 - Добавлена: математичес
21:07:05 - Добавлена: персональна
Включить уведомления
Всего: 4 | Выполнено: 1 | Осталось: 3 | Ср.приоритет: 3,2 | Выполнено: 25% | По приоритетам: [1]:1 [3]:1 [4]:1 [5]:1
Уведомления выключены

7. ВЫВОД

В ходе выполнения лабораторной работы я успешно освоил ключевые механизмы работы с делегатами и событиями в языке программирования C#. Получил практический опыт создания собственных делегатов с использованием ключевого слова delegate, понял концепцию сигнатуры метода и совместимости типов. Реализовал multicast-делегаты, которые содержат несколько методов в списке вызовов и позволяют запускать множественные обработчики одним вызовом. Создал систему уведомлений, где один multicast-делегат одновременно записывает события в журнал с временем и обновляет статусную строку. Освоил операторы += для добавления методов в цепочку и -= для их удаления, что обеспечивает гибкое управление подписками.

Освоил работу с лямбда-выражениями как компактной формой записи анонимных методов с синтаксисом =>. Применил лямбда-выражения для создания фильтров задач, функций подсчета статистики и преобразования данных. Изучил стандартные обобщённые делегаты из библиотеки .NET: Action<T> для методов без возвращаемого значения и Func<T, TResult> для методов с результатом. Успешно применил стандартные делегаты вместо собственных объявлений, что упростило код и сделало его более универсальным.

Реализовал механизм событий и паттерн проектирования "издатель-подписчик". Создал класс TaskEventArgs как наследник EventArgs для передачи данных о событии. Объявил событие с ключевым словом event, понял ключевое отличие от обычного публичного делегата: событие можно вызвать только изнутри класса, что обеспечивает инкапсуляцию. Организовал динамическое управление подписками через CheckBox "Включить уведомления", реализовал возможность подписки и отписки от событий во время выполнения программы через операторы += и -=.

Освоил технологию LINQ для работы с коллекциями через декларативные запросы. Применил методы Where для фильтрации, OrderBy и ThenByDescending для многоуровневой сортировки, Count и Average для статистики, GroupBy для группировки задач по приоритету. Реализовал расширенную статистику с вычислением среднего приоритета, процента выполненных задач и распределения по уровням приоритета. Понял, что методы LINQ активно используют делегаты и лямбда-выражения для передачи логики обработки данных.

Создал полнофункциональное графическое приложение на базе кроссплатформенного UI-фреймворка Avalonia. Разработал XAML-разметку интерфейса с Grid для организации панелей: добавление задач, список с фильтрами, журнал событий и статистика. Реализовал code-behind логику с обработчиками событий для всех элементов управления. Применил принципы разделения ответственности: TaskItem инкапсулирует данные, TaskManager управляет коллекцией и генерирует события, MainWindow отвечает за представление. Освоил паттерн Observer через механизм событий, что обеспечивает слабую связанность компонентов.

Полученные навыки работы с делегатами, событиями и лямбда-выражениями являются фундаментальными для современного программирования на C#. Освоил паттерн "издатель-подписчик" как ключевой механизм слабой связанности, который широко применяется в GUI-фреймворках и event-driven архитектурах. Научился писать компактный и выразительный код с помощью лямбда-выражений и LINQ. Эти знания станут основой для работы с асинхронным программированием, реактивными расширениями и создания масштабируемых систем на платформе .NET.